Supervyse OPF Redfish Validator User Guide

Revision 1.0
12/2024

Copyright (c) 2024, All Rights Reserved. Insyde Software.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form, or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Insyde Software Corp.

Disclaimer

Insyde Software provides this document and the programs "as is" without warranty of any kind, either expressed or implied, including, but not limited to, the implied warranties of merchantability or fitness for a particular purpose.

This document could contain technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in future revisions of this document. Insyde Software is under no obligation to notify any person of the changes.

The following trademarks are used in this document:

Insyde and InsydeH2O are registered trademarks or trademarks of Insyde Software. All other trademarks or trade names are property of their respective holders.

Contents

1. Introduction 5
   1.1 Overview 5
   1.2 Related Information 5
2. Redfish Service Validator 6
   2.1 Introduction 6
   2.2 Installation 6
   2.3 Requirements 6
   2.4 Usage 6
      2.4.1. Without a configuration file 6
      2.4.2. With a configuration file 7
   2.5 Execution Flow 12
   2.6 Generated Report 13
   2.7 Limitations 14
3. Redfish Protocol Validator 15
   3.1 Introduction 15
   3.2 Installation 15
   3.3 Requirements 15
   3.4 Usage 16
   3.5 Generated Report 17
4. Redfish Interoperability Validator 18
   4.1 Introduction 18
   4.2 Installation 18
   4.3 Requirements 18
   4.4 Usage 18
      4.4.1. Without a configuration file 18
      4.4.2. With a configuration file 19
   4.5 Execution Flow 23
   4.6 Generated Report 24

Revision History

Revision Number	Description	Author	Reviewer	Release Date
1.0	Initial release.	Cindy Liu	Victor Wu	2024-12-10

1. Introduction

This document is designed to serve a diverse audience involved in the use, testing, and development of Redfish-related validation tools. Each section caters to specific roles, ensuring that readers can quickly access the information most relevant to their needs.

1.1 Overview

This document includes the usage of the Redfish Service Validator, Redfish Protocol Validator, and Redfish Interoperability Validator.

1.2 Related Information

The tool is designed to be device-agnostic and is driven based on the Redfish specifications and schema intended to be supported by the device.

- https://github.com/DMTF/Redfish-Service-Validator
- https://github.com/DMTF/Redfish-Protocol-Validator
- https://github.com/DMTF/Redfish-Interop-Validator
- Download location of profiles: *https://github.com/opencomputeproject/HWMgmt-OCP-Profiles*

2. Redfish Service Validator

2.1 Introduction

The Redfish Service Validator is a Python3 tool for checking conformance of any "device" with a Redfish interface against Redfish CSDL schema.

- https://github.com/DMTF/Redfish-Service-Validator

2.2 Installation

From PyPI:

```
pip3 install redfish_service_validator
```

From GitHub:

```
git clone https://github.com/DMTF/Redfish-Service-Validator.git
cd Redfish-Service-Validator
python3 setup.py sdist
pip3 install dist/redfish_service_validator-x.x.x.tar.gz
```

2.3 Requirements

External modules:

- beautifulsoup4
- requests
- lxml

Install the prerequisites by running:

```
pip3 install –r requirements.txt
```

2.4 Usage

2.4.1. Without a configuration file

```
python3 RedfishServiceValidator.py –r https://<bmc_ip> -u <username> -p <password>
```

e.g. python3 RedfishServiceValidator.py -r https://192.168.9.156 -u root -p 0penBmc

```
cl@cindy-liu:/mnt/c/Users/cindy.liu/Redfish-Service-Validator$ rf_service_validator -r https://192.168.9.156 -u root -p 0penBmc
Redfish Service Validator, version 2.4.9

Writing config file to log directory
LogEntry 20
Target URI: https://192.168.9.156
authtype: Basic
collectionlimit: ['LogEntry', '20']
debugging: False
forceauth: False
ip: https://192.168.9.156
logdir: ./logs
oemcheck: True
password: ******
requestattempts: 10
requesttimeout: 10
schema_directory: ./SchemaFiles/metadata
uricheck: False
username: root
verbose: 0
Start time: 12/10/24 - 13:23:10

Setting up service...
Attempt 1 of /redfish/v1/
Response Time for GET to /redfish/v1/: 0.1356461249961285 seconds.
Attempt 1 of /redfish/v1/SessionService/Sessions
Response Time for GET to /redfish/v1/SessionService/Sessions: 0.8649957589950645 seconds.
Attempt 1 of /redfish/v1/$metadata
```

Figure 2.4.1-1 Command without a configuration file

2.4.2. With a configuration file

```
python3 RedfishServiceValidator.py –c config/example.ini
```

The following sections describe the arguments and configuration file options. The file `config/example.ini` can be used as a template configuration file.

At a minimum, the `ip`, `username`, and `password` options must be modified.

```
[Tool]
verbose =

[Host]
ip = https://192.168.9.156
username = root
password = OpenBmc
description = MySystem
forceauth = False
authtype = Basic
token =

[Validator]
payload =
logdir = ./logs
oemcheck = True
debugging = False
schema_directory = ./SchemaFiles/metadata
collectionlimit = LogEntry 20
```

Figure 2.4.2-1 Configuration File

Table 2.4.2-1 Options of Redfish Service Validator -- [Tool]

Variable	CLI Argument	Type	Definition	Example
verbose	-v	integer	Verbosity of tool in stdout. (0~3)	0

Table 2.4.2-2 compares the differences between verbose levels 0-3, using **/redfish/v1/AccountService** as an example.

Table 2.4.2-2 Comparison of Verbose Levels 0 to 3

Verbose Level	Log Content	Example Messages	Use Case
0 (Minimal)	1. Basic test info (start time, target URI, request results) 2. Error summary (pass/fail count) 3. No request/response details 4. No deep debugging info.	INFO - Attempt 1 of /redfish/v1/AccountService INFO - Response Time for GET to /redfish/v1/AccountService: 0.234s WARNING - Couldn't get schema for object, skipping OemObject OpenBMC	1. Basic validation check 2. CI/CD pipeline summary
1 (Basic Debugging)	1. Includes VERBOSE1 messages 2. Shows object properties and schema validation 3. Highlights missing	VERBOSE1 - ('Morphing Complex', 'AccountService.v1_13_0', 'ExternalAccountProvider', 'AccountService.v1_13_0') VERBOSE1 - MultiFactorAuth: value=n/a (Optional) -> Property does not exist, skipping.	1. Debugging schema validation issues 2. Investigating missing properties

	attributes or schema mismatches 4. No HTTP request/response details		
2 (Detailed Debugging)	1. Everything from verbose1 2. More VERBOSE1 messages with object morphing details 3. Shows property handling logic (e.g., optional vs required fields) 4. No HTTP request/response headers or body	VERBOSE1 - PasswordExpirationDays: value=n/a (Optional) -> Property does not exist, skipping. VERBOSE1 - LDAP: {'Authentication': {'AuthenticationType': 'UsernameAndPassword', 'Password': None, 'Username': ''}, 'ServiceEnabled': False}	1.Investigating optional fields 2. Checking LDAP/Auth property handling
3 (Full Debugging)	1. Everything from verbose2 2. Adds DEBUG messages with full HTTP request/response logs 3. Shows request headers, response headers, and response body 4. Can debug API communication issues 5. Logs response codes (e.g., 200 OK, 403 Forbidden).	**Request:** DEBUG - HTTP REQUEST (GET) for /redfish/v1/AccountService Headers: Accept: */* OData-Version: 4.0 **Response:** DEBUG - HTTP RESPONSE for /redfish/v1/AccountService Code: 200 OK Headers: Content-Type: application/json Body: {"@odata.id": "/redfish/v1/AccountService", "ServiceEnabled": true}	1. Debugging API failures 2. Checking HTTP headers and responses 3. Deep troubleshooting authentication errors

Table 2.4.2-3 Options of Redfish Service Validator -- [Host]

Variable	CLI Argument	Type	Definition	Example
ip	**-r**	**string**	**The address of the Redfish service.**	**https://192.168.9.156**
username	**-u**	**string**	**The username for authentication.**	**root**
password	**-p**	**string**	**The password for authentication.**	**OpenBmc**

description	--description	string	The description of the system for identifying logs;	MySystem
forceauth	--forceauth	boolean	Force authentication on unsecure connections.	False
authtype	--authtype	string	Authorization type. ('None', 'Basic', 'Session', or 'Token')	Basic
token	--token	string	Token when 'authtype' is 'Token'.	--token <your_token>
ext_http_proxy	--ext_http_proxy	string	URL of the HTTP proxy for accessing external sites.	--ext_http_proxy <HTTP_proxy_uri>
ext_https_proxy	--ext_https_proxy	string	URL of the HTTPS proxy for accessing external sites.	--ext_https_proxy <HTTP_proxy_uri>
serv_http_proxy	--serv_http_proxy	string	URL of the HTTP proxy for accessing the service.	--serv_http_proxy <HTTP_proxy_uri>
serv_https_proxy	--serv_https_proxy	string	URL of the HTTPS proxy for accessing the service.	--serv_https_proxy <HTTP_proxy_uri>

Table 2.4.2-4 Options of Redfish Service Validator -- [Validator]

Variable	CLI Argument	Type	Definition	Example
payload	--payload	string	The mode to validate payloads. Check out Payload Option for more description. ('Tree', 'Single', 'SingleFile', or 'TreeFile' followed by resource/filepath)	--payload Single /redfsih/v1/AccountService
logdir	--logdir	string	The directory for generated report files.	./logs
oemcheck	--nooemcheck	boolean	Whether to check OEM items on service.	True
uricheck	--uricheck	boolean	Allow URI checking on services below RedfishVersion 1.6.0.	True
debugging	--debugging	boolean	Output debug statements to text log, otherwise it only uses INFO.	False
schema directory	--schema_directory	string	Directory for local schema files.	./SchemaFiles/metadata
mockup	--mockup	string	Directory tree for local mockup files. This option enables insertion of local mockup resources to replace missing, incomplete, or incorrect implementations retrieved from the service that may hinder full validation coverage. For examples of full mockups, see the Redfish Mockup Bundle (DSP2043) at https://www.dmtf.org/sites/default/files/standards/documents/DSP2043_2024.1.zip.	--mockup <your_mockup_files_location>
collectionlimit	--collectionlimit	string	Apply a limit to collections. (Format: RESOURCE1 COUNT1 RESOURCE2 COUNT2...)	ComputerSystem 20 (limits ComputerSystemCollection to 20 links.)

requesttimeout	--requesttimeout	integer	Timeout in seconds for HTTP request waiting for response.	10
requestattempts	--requestattempts	integer	Number of attempts after failed HTTP requests.	10

•Payload Option

The `payload` option takes two parameters as strings.

The first parameter specifies how to test the payload URI given, which can be 'Single', 'SingleFile', 'Tree', or 'TreeFile'.

1. 'Single' and 'SingleFile' will test and give a report on a single resource.
2. 'Tree' and 'TreeFile' will test and give a report on the resource and every link from that resource.

The second parameter specifies a URI of the target payload to test or a filename of a local file to test.

For example:

```
--payload Single /redfish/v1/AccountService
```

Figure 2.4.2-2 Commands with Payload Option

```
cl@cindy-liu:/mnt/c/Users/cindy.liu/Redfish-Service-Validator/config$ rf_service_validator -r https://192.168.9.156 -u root -p 0penBmc --payload Single /redfish/v1/AccountService
Redfish Service Validator, version 2.4.9

Writing config file to log directory
Single /redfish/v1/AccountService
LogEntry 20
Target URI: https://192.168.9.156
authtype: Basic
collectionlimit: ['LogEntry', '20']
debugging: False
forceauth: False
ip: https://192.168.9.156
logdir: ./logs
oemcheck: True
password: ******
payload: ['Single', '/redfish/v1/AccountService']
requestattempts: 10
requesttimeout: 10
schema_directory: ./SchemaFiles/metadata
uricheck: False
username: root
verbose: 0
Start time: 12/10/24 - 10:28:48

Downloading initial schemas from online
The tool will, by default, attempt to download and store XML files to relieve traffic from DMTF/service
Unpacking schema pack... https://www.dmtf.org/sites/default/files/standards/documents/DSP8010.zip
```

2.5 Execution Flow

1.**Initial Query**:

- Query the service root resource to gather information and collect CSDL data.

2.**Resource Validation**:

- Read all referenced URIs.
- Load the resource's schema and build an expected model of properties.
- Validate each property against the schema model.
- Repeat until all resources are validated.

3.**Validation Tests**: For each property:

- Verify `@odata` properties (e.g., `@odata.id`).
- Check if the property is defined in the schema.
- Ensure the value matches the expected type (e.g., integer, string).
- Confirm if the property is mandatory or nullable.
- For strings, validate against regular expressions.
- For enumerations, ensure values are in the allowed list.
- For numbers, confirm values are within specified ranges.
- For objects, validate properties inside the object.
- For links, ensure referenced URIs match expected resource types.

2.6 Generated Report

It will generate reports in the "logs" folder, including:

- "ConformanceHtmlLog_MM_DD_YYYY_HHMMSS.txt"
- "ConformanceHtmlLog_MM_DD_YYYY_HHMMSS.html"

Here, **MM_DD_YYYY_HHMMSS** represents the timestamp (Month, Day, Year, Hour, Minute, Second).

The reports give the detailed view of the individual properties checked, with the Pass/Fail/Skip/Warning status for each resource checked for conformance.

```
cl@cindy-liu:/mnt/c/Users/cindy.liu/Redfish-Service-Validator/logs$ ls
ConfigFile_12_05_2024_170606.ini  ConformanceHtmlLog_12_05_2024_170606.html  ConformanceLog_12_05_2024_170606.txt
```

Figure 2.6-1 The Report Generation under "logs" folder

Test Summary

Description: MySystem
System: https://192.168.11.243

errorInvalidReferenceObject: 39	**invalidPropertyValue:** 11
metadataNamespaces: 2678	**pass:** 10459
passAction: 42	**passGet:** 723
passRedfishUri: 548	**repeat:** 6
skipNoSchema: 39	**skipOptional:** 19674
unverifiedAdditional.complex: 29	**warnDeprecated:** 23
warningPresent: 26	**warnings:** 56

39 errorInvalidReferenceObject errors in /redfish/v1/Managers/bmc

authtype: Basic	**certificatebundle:** None
certificatecheck: False	**collectionlimit:** {'LogEntry': 20}
config: None	**configuri:** https://192.168.11.243
debugging: False	**ext_http_proxy:**
ext_https_proxy:	**forceauth:** False
logdir: ./logs	**metadatafilepath:** ./SchemaFiles/metadata
mockup:	**oemcheck:** True
payload: None	**requestattempts:** 10
requesttimeout: 10	**schema_directory:** ./SchemaFiles/metadata
serv_http_proxy:	**serv_https_proxy:**
token: False	**uricheck:** True
username: root	**usessl:** True
verbose: 0	

Figure 2.6-2 The Generated Report of Redfish Service Validator (1)

/redfish/v1/ (response time: 0:00:00.018235) ServiceRoot

/redfish/v1/ (response time: 0:00:00.018235) Show Results Show Payload	**Context:** /redfish/v1/$metadata#ServiceRoot.ServiceRoot **File Origin:** ServiceRoot_v1.xml **Resource Type:** ServiceRoot.v1_14_0.ServiceRoot	GET Success HTTP Code (200)	**pass:** 28 **passGet:** 1 **passRedfishUri:** 1 **skipNoSchema:** 2 **skipOptional:** 21 **warningPresent:** 1 **warnings:** 1

Property Name	Value	Type	Exists	Result
@odata.id	/redfish/v1	odata	Exists	PASS
@odata.type	#ServiceRoot.v1_14_0.ServiceRoot	odata	Exists	PASS
ServiceIdentification	-	string	No	Optional
ServiceConditions	-	link to: ServiceConditions	No	Optional
ComponentIntegrity	-	link to: ComponentIntegrityCollection	No	Optional
RegisteredClients	-	link to: RegisteredClientCollection	No	Optional
LicenseService	-	link to: LicenseService	No	Optional
KeyService	-	link to: KeyService	No	Optional
Cables	-	link to: CableCollection	No	Optional
Storage	-	link to: StorageCollection	No	Optional
AggregationService	-	link to: AggregationService	No	Optional

Figure 2.6-3 The Generated Report of Redfish Service Validator (2)

2.7 Limitations

The Redfish Service Validator only performs GET operations on the service. Below are certain items that are not in scope for the tool.

- Other HTTP methods, such as PATCH, are not covered.
- Query parameters, such as $top and $skip, are not covered.
- Multiple services are not tested simultaneously.

3. Redfish Protocol Validator

3.1 Introduction

The Redfish Protocol Validator tests the HTTP protocol behavior of a Redfish service to validate that it conforms to the Redfish Specification.

•https://github.com/DMTF/Redfish-Protocol-Validator

3.2 Installation

From PyPI:

```
pip3 install redfish_protocol_validator
```

From GitHub:

```
git clone https://github.com/DMTF/Redfish-Protocol-Validator.git
cd Redfish-Protocol-Validator
python3 setup.py sdist
pip3 install dist/redfish_protocol_validator-x.x.x.tar.gz
```

3.3 Requirements

The Redfish Protocol Validator requires Python3.

Required external packages:

- aenum
- colorama
- pyasn1
- pyasn1-modules
- requests
- sseclient-py
- urllib3

Install the prerequisites by running:

```
pip3 install -r requirements.txt
```

3.4 Usage

```
python3 rf_protocol_validator.py -r https://<bmc_ip> -u <username> -p <password>
```

e.g. python3 rf_protocol_validator.py -r http://192.168.9.156 -u root -p 0penBmc

Table 3.4-1 Options of Configuration File

Variable	CLI Argument	Type	Definition	Example
ip	**-r**	**string**	**The address of the Redfish service.**	**http://192.168.9.156**
user	**-u**	**string**	**The username for authentication.**	**root**
password	**-p**	**string**	**The password for authentication.**	**0penBmc**
log level	--log-level	string	The logging level. (Default: WARNING)	WARNING
report directory	--report-dir	string	The directory for generated report files. (Default: "reports")	reports
report type	--report-type	string	The type of report to generate: html, tsv, or both. (Default: both)	both
	--avoid-http-redirect		Avoid attempts to generate HTTP redirects for services that do not support HTTP.	--avoid-http-redirect
	--no-cert-check		Disable verification of host SSL certificates.	--no-cert-check
	--ca-bundle	string	The file or directory containing trusted CAs.	--ca-bundle <your_CAs_location>

3.5 Generated Report

It will generate reports in the "reports" folder, including:

- "RedfishProtocolValidationReport_MM_DD_YYYY_HHMMSS.tsv"
- "RedfishProtocolValidationReport_MM_DD_YYYY_HHMMSS.html"

Here, **MM_DD_YYYY_HHMMSS** represents the timestamp (Month, Day, Year, Hour, Minute, Second).

The reports give the detailed view of the individual properties checked, with the Pass/Fail/Skip/Warning status for each resource checked for conformance.

```
cl@cindy-liu:/mnt/c/Users/cindy.liu/Redfish-Protocol-Validator/reports$ ls
RedfishProtocolValidationReport_12_05_2024_170410.html  RedfishProtocolValidationReport_12_05_2024_170410.tsv  results.json
```

Figure 3.5-1 The Report Generation under "reports" folder

System: https://192.168.10.231/redfish/v1/, User: root, Password: ****** Product: P3809 Manufacturer: N/A, Model: OpenBmc, Firmware version: 35.24.44.3237**
Results Summary Pass: 420, Warning: 0, Fail: 4, Not tested: 32

Figure 3.5-2 The Generated Report of Redfish Protocol Validator (1)

PROTO_HTTP_SUPPORTED_METHODS: "HTTP methods: Services shall support the POST, GET, PATCH, and DELETE HTTP methods."				
Result	**Method**	**Status code**	**URI**	**Message**
PASS	GET	200		GET supported
PASS	POST	201		POST supported
PASS	PATCH	200		PATCH supported
PASS	DELETE	200		DELETE supported
PROTO_HTTP_UNSUPPORTED_METHODS: "HTTP methods: Services shall return the HTTP 405 Method Not Allowed status code or the HTTP 501 Not Implemented status code for unsupported and undocumented methods."				
Result	**Method**	**Status code**	**URI**	**Message**
PASS	DELETE	405	/redfish/v1/	Test passed
PASS	PATCH	405	/redfish/v1/	Test passed
PASS	POST	405	/redfish/v1/	Test passed
PASS	FAKEMETHODFORTEST	405	/redfish/v1/	Test passed
PROTO_JSON_ACCEPTED: "Media types: Receivers shall not reject a JSON-encoded message, and shall offer at least one JSON-based response representation."				
Result	**Method**	**Status code**	**URI**	**Message**
PASS	POST	201	/redfish/v1/SessionService/Sessions	Test passed
PASS	POST	201	/redfish/v1/AccountService/Accounts	Test passed
PASS	PATCH	200	/redfish/v1/AccountService/Accounts/rfpv864c	Test passed

Figure 3.5-3 The Generated Report of Redfish Protocol Validator (2)

4. Redfish Interoperability Validator

4.1 Introduction

The Redfish Interop Validator is a Python 3 tool that verifies whether a service complies with a specified interoperability profile. It follows the DMTF Redfish Profile schema and works with any Redfish-compliant device, regardless of hardware. This tool ensures that a service meets vendor specifications, enabling compatibility with vendor systems and tools.

•https://github.com/DMTF/Redfish-Interop-Validator

4.2 Installation

From PyPI:

```
pip3 install redfish_interop_validator
```

From GitHub:

```
git clone https://github.com/DMTF/Redfish-Interop-Validator.git
cd Redfish-Interop-Validator
python3 setup.py sdist
pip3 install dist/redfish_interop_validator-x.x.x.tar.gz
```

4.3 Requirements

External modules:

- beautifulsoup4
- requests
- lxml
- jsonschema

Install the prerequisites by running:

```
pip3 install -r requirements.txt
```

4.4 Usage

4.4.1. Without a configuration file

```
python3 RedfishInteropValidator.py -r https://<bmc_ip> -u <username> -p <password> profile
[profile ...]
```

e.g. python3 RedfishInteropValidator.py -r https://192.168.9.156 -u root -p 0penBmc testprofiles/ChassisType_1.json

```
cl@cindy-liu:/mnt/c/Users/cindy.liu/Redfish-Interop-Validator$ rf_interop_validator -r https://192.168.9.156 -u root -p 0penBmc test-profiles/ChassisType_1.json
Redfish Interop Validator, version 2.2.7

Writing config file to log directory
LogEntry 20
Target URI: https://192.168.9.156
authtype: Basic
collectionlimit: ['LogEntry', '20']
debugging: False
forceauth: False
ip: https://192.168.9.156
logdir: ./logs
oemcheck: True
online_profiles: True
password: ******
profile: ['test-profiles/ChassisType_1.json']
username: root
verbose: 0
warnrecommended: False
writecheck: False
Start time: 12/10/24 - 11:47:06

Setting up service...
Redfish Version of Service: 1.17.0
Description of service: My Target System Root Service, version 1.17.0, e04cc84b-11bb-5dad-9276-65137f1bf1ee

Profile Hashes:
profile: ChassisType5 1.0.0, dict md5 hash: 423d7ecc41119ce5fa9d406774caf1ac
```

Figure 4.4.1-1 Command without a configuration file

4.4.2. With a configuration file

python3 RedfishInteropValidator.py -c config/example.ini profile [profile ...]

e.g. python3 RedfishInteropValidator.py -c config/example.ini test-profiles/ChassisType_1.json

```
[Tool]
verbose =

[Host]
ip = https://192.168.9.156
username = root
password = 0penBmc
description = MySystem
forceauth = False
authtype = Basic
token =

[Validator]
payload =
logdir = ./logs
oemcheck = True
online_profiles = True
debugging = False
collectionlimit = LogEntry 20
```

Figure 4.4.2-1 Configuration file

Modify the config/example.ini file to enter the system details under below section

Table 4.4.2-1 Options of Configuration File -- [Tool]

Variable	CLI Argument	Type	Definition	Example
verbose	-v	integer	Verbosity of tool in stdout. (0~3)	0

Table 4.4.2-2 compares the differences between verbose levels 0-3, using **/redfish/v1/Systems/system** as an example.

Table 4.4.2-2 Comparison of Verbose Levels 0 to 3

Verbose Level	Log Content	Example Messages	Use Case
0 (Minimal)	1. Only basic test info is logged (start time, target URI, result). 2. Includes error summary (pass/fail count). 3. No request/response details. 4. No in-depth debugging.	INFO - GET request to /redfish/v1/Systems/system started INFO - Response Time: 0.234s for GET /redfish/v1/Systems/system WARNING - Schema for system not found, skipping	1. Basic validation. 2. Check request/response timing. 3. Basic error tracking.
1 (Basic Debugging)	1. Logs more details about object properties. 2. Shows missing attributes or schema mismatches. 3. No HTTP request/response details.	VERBOSE1 - Fetching system properties for /redfish/v1/Systems/system VERBOSE1 - Missing schema attribute: 'Memory' for system VERBOSE1 - Property 'PowerState' validated successfully	1. Identifying missing or invalid schema attributes. 2. Tracking object properties.
2 (Detailed Debugging)	1. Includes everything from verbose level 1.	VERBOSE1 - PowerState: value=On (Required) -> Property exists	1. Investigating detailed property handling 2. Checking required vs optional fields

	2. Shows detailed property handling logic. 3. Investigates optional vs required fields. 4. No HTTP request/response headers or body.	VERBOSE1 - Storage: {'Type': 'SSD', 'SizeGB': 512} VERBOSE1 - Memory: Optional -> Skipping check for 'SizeMB'	
3 (Full Debugging)	1. Logs everything from verbose level 2 2. Includes full HTTP request and response logs. 3. Logs request/response headers and body. 4. Response codes and communication details.	**Request:** DEBUG - HTTP REQUEST (GET) for /redfish/v1/Systems/system Headers: Accept: application/json OData-Version: 4.0 **Response:** DEBUG - HTTP RESPONSE for /redfish/v1/Systems/system Code: 200 OK Headers: Content-Type: application/json Body: {"PowerState": "On", "Memory": {"SizeGB": 64}, "Storage": {"Type": "SSD", "SizeGB": 512}}	1. Debugging API failures. 2. Full HTTP request/response analysis. 3. Troubleshooting authentication and communication errors.

Table 4.4.2-3 Options of Configuration File -- [Interop]

CLI Argument	Type	Definition	Exaxmple
profile	**string**	**The name of the testing profile (mandatory).**	**test-profile/ChassisType_1.json**
--schema	string	The filepath and name of the schema file to verify the format of the profile.	--schema <your_schema_file_location>

If you want to download the profile, please visit this URL: https://github.com/opencomputeproject/HWMgmt-OCP-Profiles

Table 4.4.2-4 Options of Configuration File -- [Host]

Variable	CLI Argument	Type	Definition	Example
ip	**-r**	**string**	**The address of the Redfish service.**	**http://192.168.9.156**
username	**-u**	**string**	**The username for authentication.**	**root**
password	**-p**	**string**	**The password for authentication.**	**OpenBmc**
description	--description	string	The description of the system for identifying logs.	MySystem
forceauth	--forceauth	boolean	Force authentication on unsecure connections.	False
authtype	--authtype	string	Authorization type. (None|Basic|Session|Token)	Basic
token	--token	string	Token when 'authtype' is 'Token'.	--token <your_token>

Table 4.4.2-5 Options of Configuration File -- [Validator]

Variable	CLI Argument	Type	Definition	Example
payload	--payload	string	The mode to validate payloads. check out Payload Option for more description. ([Tree, Single, SingleFile, TreeFile] followed by resource/filepath)	--payload Single /redfsih/v1/AccountService
logdir	--logdir	string	The directory for generated report files.	./logs
oemcheck	--nooemcheck	boolean	Whether to check OEM items on service.	True

online_profiles	--no_online_profiles	boolean	Whether to download online profiles.	True
debugging	--debugging	boolean	Output debug statements to text log, otherwise it only uses INFO.	False
required_profiles_dir	--required_profiles_dir	string	Option to set the root folder of required profiles.	--required_profiles_dir <your_required_profiles_dir_location>
collectionlimit	--collectionlimit	string	Apply a limit to collections. (Format: RESOURCE1 COUNT1 RESOURCE2 COUNT2...)	ComputerSystem 20 (limits ComputerSystemCollection to 20 links.)

•Payload options

The payload option takes two parameters as "option uri"

(Single, SingleFile, Tree, TreeFile) How to test the payload URI given.

Single tests will only give a report on a single resource, while Tree will report on every link from that resource

([Filename], [uri])

URI of the target payload, or filename of a local file.

4.5 Execution Flow

1. **Start with Service Root**:
   - Query the service root URI to retrieve device info, supported resources, and links.
   - Validate the response against the profile (if provided).
2. **Traverse Resources**:
   - Explore all collections and navigation properties.
   - Query each resource URI and validate its properties against the profile.
   - Repeat until all resources are covered.
3. **Validation Tests**:
   - **MinVersion**: Ensure the resource version meets or exceeds the profile's minimum version.
   - **MinCount**: Verify collection size meets the minimum required.
   - **ReadRequirement**: Check if properties/resources are mandatory or recommended.
   - **Members**: Validate "Members" properties, including size checks.
   - **MinSupportedValues**: Ensure enumerations match the profile's supported values.
   - **Writeable**: Confirm properties support write operations if required.

- **Comparison**: Validate properties using conditions like `AnyOf`, `AllOf`, `GreaterThan`, `Equal`, etc.
- **ConditionalRequirements**: Perform specific tests if conditions (e.g., type hierarchy, comparisons) are true.
- **ActionRequirements**: Validate requirements for specific actions, including nullability and mandatory checks.

4.**Collections**:

- Validate all items in a collection, checking each property's compliance.

Note: Unlike the Service Validator, this tool skips listing problematic resources, assuming such issues are handled elsewhere.

4.6 Generated Report

It will generate reports in the "logs" folder, including:

- "InteropLog_MM_DD_YYYY_HHMMSS.txt"
- "InteropHtmlLog_MM_DD_YYYY_HHMMSS.html"

Here, **MM_DD_YYYY_HHMMSS** represents the timestamp (Month, Day, Year, Hour, Minute, Second).

The reports give the detailed view of the individual properties checked, with the Pass/Fail/Skip/Warning status for each resource checked for conformance.

```
cl@cindy-liu:/mnt/c/Users/cindy.liu/Redfish-Interop-Validator/logs$ ls
ConfigFile_12_05_2024_165957.ini        InteropHtmlLog_12_06_2024_161841.html  InteropLog_12_06_2024_161627.txt
```

Figure 4.6-1 The Report Generation under "logs" folder

Description: My Target System Root Service, version 1.17.0, e04cc84b-11bb-5dad-9276-65137f1bf1ee
System: https://192.168.9.151

Profile: ['test-profiles/Systems_Profile.v1_0_0.json']
Schema: None

authtype: Basic, certificatebundle: None, certificatecheck: False, collectionlimit: {'LogEntry': 20}
config: None, configuri: https://192.168.9.151, debugging: False, forceauth: False
ip: https://192.168.9.151, logdir: ./logs, oemcheck: True, online_profiles: True
payload: None, required_profiles_dir: None, timeout: 10, token: None
username: root, usessl: True, verbose: 0, warnrecommended: False
writecheck: False

inaccessibleResource: 4	**notTested:** 1
pass: 21	**passGet:** 1296
totaltests: 20	**warningPresent:** 5

Expand All | Collapse All

/redfish/v1/Systems/system (response time: 0.515579) ComputerSystem

/redfish/v1/Systems/system (response time: 0.515579) Show Results	**Context:** /redfish/v1/$metadata#ComputerSystem.ComputerSystem **File Origin:** None **Resource Type:** #ComputerSystem.v1_23_0.ComputerSystem	GET Success	**notTested:** 1 **pass:** 16 **passGet:** 1 **totaltests:** 18 **warningPresent:** 1

Property Name	Value	Expected	Actual	Result
MinVersion	1.1.0	<=	#ComputerSystem.v1_23_0.ComputerSystem	PASS
SerialNumber.ReadRequirement	Mandatory	Must Exist	Exists	PASS
Manufacturer.ReadRequirement	Mandatory	Must Exist	Exists	PASS
SystemType.ReadRequirement	Mandatory	Must Exist	Exists	PASS
SystemType.Comparison	['Composed', 'OS', 'Physical']	Equal	Physical	PASS

Figure 4.6-2 The Generated Report of Redfish Interoperability Validator